

Analyse & Propositions : Amélioration du Moteur de Recherche d'Infocité

Document d'étude technique et fonctionnelle — Mai 2026

Ce document présente une analyse technique et fonctionnelle visant à transformer le moteur de recherche actuel d'Infocité en un système de recherche approfondi et intelligent. Suite aux retours du COPIL et à la spécification de notre écosystème (**Google for Education / Google Workspace**), nous orientons l'architecture vers une intégration poussée des services Google.

1. Diagnostic de l'existant

DIAGNOSTIC

Fonctionnement actuel

Actuellement, la recherche globale dans Infocité (gérée par `SearchResultManager`) utilise des requêtes SQL basiques basées sur l'opérateur `LIKE` :

```
'foundPosts' => Post::query()
->where('title', 'like', "%$this->searchedStr%")
->orWhere('content', 'like', "%$this->searchedStr%")
```

Limites identifiées :

- Aucune tolérance aux fautes (typos)** : Si l'utilisateur fait une faute de frappe ou cherche un singulier alors que le texte contient un pluriel, aucun résultat n'est trouvé.
- Absence de recherche sémantique** : Impossible de trouver un article sur les "nouveaux arrivants" si l'utilisateur saisit le terme "onboarding" (à moins que le mot exact ne soit présent dans l'article).
- Contenu Google Drive invisible** : De nombreux articles contiennent des liens (URLs) vers des documents stockés sur le Google Drive de l'établissement (Google Docs, Sheets,

PDFs, etc.). Le contenu textuel de ces documents n'est actuellement pas indexé, ce qui limite considérablement la portée de la recherche.

2. Propositions d'amélioration du moteur interne

Pour rendre la recherche la plus approfondie possible, nous proposons trois niveaux d'amélioration progressive du moteur de recherche interne.

NIVEAU 1 Recherche textuelle avancée (Full-Text Search)

Technologie proposée : Intégration de **Laravel Scout** couplé à **Meilisearch** (open-source, léger, extrêmement rapide et facile à déployer localement).

Fonctionnalités apportées :

- **Tolérance aux fautes d'orthographe (typo-tolerance) :** Gère automatiquement les fautes légères.
- **Racinisation (stemming) et lemmatisation :** Associe les mots dérivés (ex: chercher "gestion" trouvera "gérer", "gestionnaire").
- **Pondération des champs :** Attribuer plus d'importance aux mots trouvés dans le titre qu'à ceux trouvés dans le corps du texte.
- **Recherche par expressions exactes ou exclusion** (ex: "sécurité informatique" - externe).

NIVEAU 2 Recherche sémantique par vecteurs (Vector Search)

Technologie proposée : Vectorisation (embeddings) des articles via un modèle d'IA (ex: API Google Gemini Embeddings) stockés dans une base de données vectorielle (extension `pgvector` pour PostgreSQL, ou une base dédiée comme Qdrant/Chroma).

Fonctionnalités apportées :

- **Compréhension conceptuelle** : Associe des concepts proches (ex: "télétravail" et "travail à distance" ou "onboarding" et "intégration").
- **Recherche multilingue naturelle**.

3. Analyse de la recherche dans le Google Drive de l'établissement

Le COPIL souhaite étendre la recherche aux contenus des documents du Google Drive de l'établissement référencés via des URLs dans les articles. **C'est tout à fait réalisable**, et nous avons analysé deux grandes stratégies pour y parvenir.

OPTION A L'indexation asynchrone (Pipeline classique d'indexation)

Dans ce modèle, le système extrait et indexe le texte des documents Google Drive en tâche de fond.

Schéma du pipeline :



| Liaison du texte indexé à l'ID de l'Article
└ Non → Fin du traitement

Fonctionnement technique :

1. **Extraction d'URLs** : Lors de l'enregistrement d'un article, un **Observer** Eloquent ou un **Job** asynchrone scanne le contenu HTML pour identifier les liens vers `drive.google.com`.
2. **Extraction du contenu via API Google Drive** : À l'aide de comptes de service sécurisés (API Google Drive), le serveur récupère les fichiers.
 - Les documents de traitement de texte (Google Docs) et tableurs sont exportés au format texte brut via l'API Google Drive.
 - Les fichiers PDF sont lus par des bibliothèques d'extraction de texte (avec OCR si ce sont des PDFs scannés).
3. **Indexation** : Le contenu textuel extrait est stocké en base locale ou dans le moteur de recherche (Meilisearch), associé à l'article d'origine.
4. **Recherche** : Lorsqu'un utilisateur effectue une recherche, le moteur cherche à la fois dans le corps de l'article et dans le texte des documents Google Drive associés.

Avantages :

- **Vitesse instantanée (<100ms)** : La recherche s'effectue sur un index pré-calculé en local.
- **Pertinence unifiée** : Les documents Google Drive ressortent naturellement dans les résultats de recherche d'articles.

Inconvénients :

- **Décalage de synchronisation** : Si le fichier sur le Drive est modifié par un collaborateur, la recherche ne verra la mise à jour qu'à la prochaine ré-indexation (cron journalier ou webhook d'écoute du Google Drive).

OPTION B L'Agent IA dédié à la recherche (Approche RAG dynamique)

L'utilisation d'un **Agent IA dédié** est une approche extrêmement puissante qui apporte une dimension qualitative inédite.

Fonctionnement technique :

1. **Interface conversationnelle** : À côté de la barre de recherche classique, on propose un "Assistant IA d'établissement".
2. **Compréhension de la question** : L'utilisateur pose une question en langage naturel (ex: "*Quelles sont les consignes de sécurité incendie et où est le point de rassemblement d'après les documents ?*").
3. **Recherche & Récupération (RAG)** :
 - L'Agent IA commence par faire une recherche sémantique/textuelle pour trouver les articles liés.
 - Il identifie les documents Google Drive liés à ces articles.
 - Il appelle des fonctions d'API Google Drive pour aller lire dynamiquement (ou depuis un cache) le texte de ces documents spécifiques.
4. **Raisonnement & Synthèse** : L'Agent IA analyse l'ensemble des informations, formule une réponse rédigée sur mesure, et cite précisément les sources (ex: "*Le point de rassemblement est au parking Nord (voir Guide de Sécurité lié dans l'article de la Rubrique RH)*").

Avantages :

- **Intelligence supérieure** : Il ne donne pas seulement des liens, il apporte une **réponse synthétisée et structurée** à partir de plusieurs documents complexes.
- **Fraîcheur des données** : Si l'agent récupère le document en direct, l'information est toujours parfaitement à jour.

Limites techniques majeures de l'Agent IA en temps réel :

- **Latence élevée** : L'appel à l'API LLM et la lecture en temps réel de documents externes prend du temps (généralement entre 5 et 15 secondes). Ce n'est pas adapté pour une recherche instantanée dans une barre de recherche standard.
- **Coûts de token** : Envoyer des pages entières de documents Google Drive à chaque recherche d'un utilisateur peut rapidement générer des coûts d'API importants (selon le volume de recherche).

4. Notre Recommandation : L'Architecture Hybride

Pour concilier la **vitesse** de recherche et l'**intelligence** d'un Agent IA, nous recommandons une **architecture en deux temps**.

VOLET 1 L'Indexation asynchrone pour la recherche instantanée (Meilisearch + API Google Drive)

- **Pour qui** : L'utilisateur qui tape rapidement un mot-clé dans la barre de recherche.
- **Résultat** : Des résultats instantanés (<100ms) avec des articles, des applications, et les documents Google Drive liés qui contiennent le mot-clé, avec surbrillance des passages pertinents.

VOLET 2 L'Assistant IA "Copilote" pour les recherches approfondies (Agent IA / RAG)

- **Pour qui** : L'utilisateur qui a une question complexe ou qui ne trouve pas facilement via la recherche classique.
- **Résultat** : Un chatbot d'établissement qui s'appuie sur la base d'indexation locale pour cibler les bons documents, puis utilise un LLM (comme Gemini) pour lire, analyser et rédiger une réponse synthétique parfaite.

5. Focus sur la Sécurité & Habilitations (Le point critique)

⚠ La sécurité est le défi majeur de cette fonctionnalité.

Si un article est public sur Infocité mais contient un lien vers un document Google Drive confidentiel (ex: données stratégiques, fiches budgétaires), le moteur de recherche ne doit absolument pas divulguer le contenu de ce document aux utilisateurs non autorisés.

SÉCURITÉ

Approches possibles & Validation Technique

L'analyse du code montre que l'**authentification unique Google OAuth 2.0 est déjà intégrée à Infocité**. Cela simplifie considérablement la mise en œuvre de la sécurité :

- **Utilisateurs connectés via Google Workspace (Majorité)** : Leur jeton d'accès (Access Token) actif en session sera utilisé pour interroger l'API Google Drive en temps réel. Si l'utilisateur n'a pas explicitement accès au document sur le Drive de l'établissement, celui-ci est écarté des résultats ou ignoré par l'Agent IA (sécurité maximale garantie à 100%).
- **Utilisateurs "classiques" (Compte/MDP - Connexion Ponctuelle/Externe)** : N'ayant pas de jeton Google actif, le système adoptera une stratégie stricte de "Zero Trust" et masquera systématiquement tous les documents Google Drive de leurs résultats de recherche. Ils conserveront uniquement l'accès aux articles et applications internes d'Infocité selon la hiérarchie classique de droits.

Recommandation validée :

Mise en place du **filtrage dynamique par droits OAuth utilisateur** pour les usagers Workspace, doublé d'une exclusion systématique pour les comptes classiques, assurant ainsi une étanchéité parfaite des informations confidentielles sans alourdir l'administration.

6. Exploitation d'un double système de Tags (Taxonomie)

Pour enrichir l'expérience de recherche et le ciblage des contenus, nous proposons d'exploiter un double mécanisme de classification par tags, combinant les besoins éditoriaux et l'organisation personnelle des usagers.

TAGS ÉDITORIAUX

Nuage de mots-clés de l'article (`posts.tags`)

Concept : Permettre à l'éditeur, lors de la création ou modification d'un article, d'associer un ensemble de mots-clés structurés (libellés courts).

- **Interfaçage côté usage** : Intégration d'un composant moderne de saisie de tags (ex: `choices.js` ou composant Livewire réactif) sous l'éditeur d'article. L'auteur saisit un mot, appuie sur "Entrée" ou "Virgule" pour l'ajouter visuellement au nuage.
- **Exploitation par le moteur de recherche** :
 - Ces tags sont enregistrés dans un champ `tags` (format JSON ou chaîne normalisée) de la table `posts`.
 - Dans l'index de recherche (Meilisearch), le champ `tags` bénéficie d'une **pondération prioritaire (boost)**. Un mot-clé correspondant exactement à un tag fait immédiatement remonter l'article en tête de liste, avant même la recherche dans le titre ou le contenu.

TAGS PERSONNELS

Classification utilisateur (`post_user.tags`)

Concept : Exploiter le champ `tags` existant (mais actuellement inactif) de la table pivot `post_user` pour permettre au lecteur de classer les articles de son choix.

- **Interfaçage côté usage** :
 - Sur la fiche d'un article, à côté des boutons de favoris ou de notation, un bouton "Ajouter un libellé" ouvre une petite interface épurée permettant de saisir ou choisir un tag personnel (ex: "À lire", "Projet X", "Urgent").
 - Un espace "Mon classeur" ou une section dédiée dans "Mes Favoris" regroupe les articles par tags personnels.
- **Exploitation par le moteur de recherche** :
 - L'utilisateur peut filtrer ses requêtes de recherche spécifiquement au sein de ses propres tags (ex: recherche du mot "*budget*" restreinte aux articles qu'il a taggués "*Compta*").
 - L'Agent IA s'appuie sur ces tags pour mieux cerner le contexte des requêtes de synthèse de l'utilisateur (ex: "*Fais-moi un résumé de mes articles taggués 'Sécurité'*").

7. Prochaines étapes suggérées pour la phase de spécification

Suite aux validations techniques sur la sécurité et à l'introduction du double système de tags, voici les points restants à arbitrer :

1. **Validation de l'interfaçage des tags** : Valider l'ergonomie de l'édition du nuage de tags côté auteur, et le positionnement du bouton d'ajout de libellé côté lecteur.
2. **Choix de l'infrastructure IA** : Souhaitez-vous utiliser des APIs cloud managées (ex: Google Cloud Vertex AI / Gemini, garantissant une confidentialité stricte des données d'établissement) ou une solution hébergée localement ?